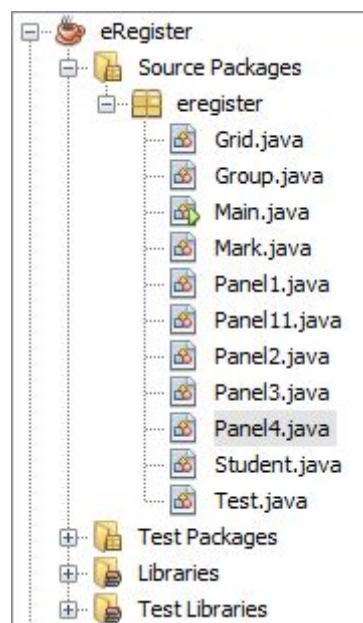


Criterion C: Development

Introduction

I have used 10 user-defined classes: 6 to create panels and 4 to store data records. In the latter case I have used the following classes: Group, Student, Test, Mark. In the main class there are the tests and groups arrays. They are initialized once the user successfully logs in with the data from the files. The 2 most important methods in the main class are the `changePanel(JButton, JPanel, JPanel)` and `update()` methods. The former adds `actionListeners` to all buttons in all panels. Once the parameter `JButton` is clicked, the first `JPanel` is removed from the `JFrame` and the second one is added. At the click of every button the `update()` method is invoked that updates the appearance of all components in all panels to match newly inputted data. Also, for certain `JButtons`, the `changePanel` method performs additional actions. After clicking the “logout and save” button in the main menu, the data is saved back to file.



Techniques used

1. User defined objects
2. Nested loops
3. Flags
4. Complex selection
5. File i/o
6. Additional libraries

1. User-defined objects

One of the user-defined classes is the Panel1 class.

```
1  ...5 lines
6  package eregister;
7
8  import ...8 lines
16
17  /**...6 lines */
23  public class Panel1 extends JPanel {
24
25      public JLabel[] label = new JLabel[3];
26      public JTextField field1 = new JTextField();
27      public JComboBox[] combol = new JComboBox[2];
28      public JButton[] button1 = new JButton[3];
29      public JLabel label2 = new JLabel("group:");
30      public JTextField field2 = new JTextField();
31
32      public Panel1() {
33          setLayout(new FlowLayout(FlowLayout.CENTER, 30, 30));
34          setPreferredSize(new Dimension(360, 300));
35          button1 = new JButton[]{new JButton("remove"), new JButton("return"), new JButton("OK")};
36          label = new JLabel[]{new JLabel("name:"), new JLabel("subject:"), new JLabel("group:")};
37          field1.setPreferredSize(new Dimension(200, 30));
38          field2.setPreferredSize(new Dimension(200, 30));
39          combol = new JComboBox[]{new JComboBox(), new JComboBox()};
40          combol[0].setPreferredSize(new Dimension(200, 30));
41          combol[1].setPreferredSize(new Dimension(200, 30));
42          add(label[0]);
43          add(field1);
44          add(label[1]);
45          add(combol[0]);
46          add(label[2]);
47          add(combol[1]);
48          add(button1[0]);
49          add(button1[1]);
50          add(button1[2]);
51      }
52  }
53
```

It extends the JPanel class to limit the amount of code written. As attributes, it has Java Swing components, which are specified in more detail in the constructor. Because FlowLayout is used, padding is specified as 30 units, the dimensions of the panel and components are explicitly stated and the components are added in specific order. There are 4 instances of Panel1 in the main class.

```

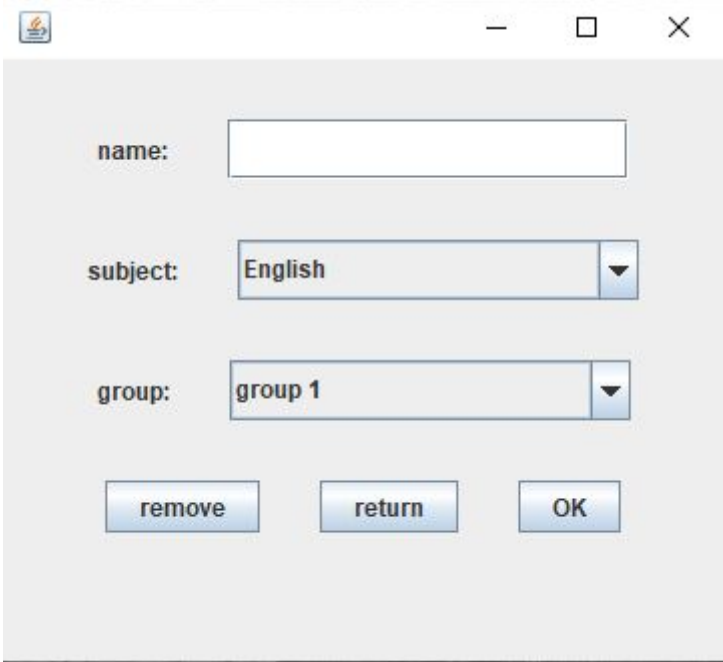
1  ⊞ ...5 lines
6  package eregister;
7
8  ⊞ import ...22 lines
30
31 ⊞ /**...4 lines */
35 public class Main {
36
37 ⊞ /** ...2 lines */
39     public static String[] subjects;
40     public static Group[] groups = new Group[20];
41     public static int groupCounter;
42     public static Test[] tests = new Test[100];
43     public static int testCounter;
44     public static int chosenTest = -1;
45     public static Student chosenStudent;
46     public static boolean csFlag = false;
47     public static long day;
48     public static int period;
49     public static String[] gNames;
50     public static String[] tNames;
51     public static String temp1 = "default";
52     public static String login = "admin";
53     public static String password = "password";
54     public static int num1;
55     public static int num2;
56     public static int var = -1;
57     public static boolean value = false;
58     public static int chosenGroup = -1;
59     public static byte[] lSalt = new byte[16];
60     public static byte[] pSalt = new byte[16];
61
62     public static JFrame frame = new JFrame();
63     public static Panell window3 = new Panell(); //calendar edit
64     public static Panell window5 = new Panell(); //marks add marks for a test
65     public static Panell window6 = new Panell(); //marks view/edit all marks
66     public static Panell window7 = new Panell(); //marks add/edit
67     public static Panel2 window8 = new Panel2(); //main menu
68     public static Panell1 window9 = new Panell1(); //marks panel menu
69     public static Panel3 window16 = new Panel3(); //login panel
70     public static Panel4 window17 = new Panel4(); //marks 5-student panel
71     public static Grid window27 = new Grid(true, false, false); //attendance panel
72     public static Grid window28 = new Grid(false, false, true); //calendar panel
73     public static Grid window29 = new Grid(false, true, false); //marks grid
74     public static Panel3 window31 = new Panel3(); //attendance student choice panel
75

```

Later, in the method `initWindows()`, each instantiation is customized by renaming, removing or otherwise changing the components.

```
865 public static void initWindows() {  
866  
867     // Panell  
868     window3.label[2].setText("group:");  
869     window3.label2.setText("type group name:");  
870     window3.combol[0].setModel(new DefaultComboBoxModel(subjects));  
}
```

window3 looks as presented below.



The screenshot shows a Java Swing window titled "window3". The window has a standard title bar with minimize, maximize, and close buttons. The main content area has a light gray background. It contains three labels: "name:", "subject:", and "group:". The "name:" label is followed by a text input field. The "subject:" label is followed by a JComboBox with "English" selected. The "group:" label is followed by a JComboBox with "group 1" selected. At the bottom, there are three buttons: "remove", "return", and "OK".

The array `subjects` has thus been set as the model of the JComboBox `window3.combol[0]`.

2. Nested loops

Nested loops have been used in file i/o, and to load the contents of the attendance and calendar panel. Let's focus on the method `listeners()`, which performs the latter.

```
113 //load tests in the calendar test edition screen
114 public static void listeners() {
115     ActionListener listener2 = new ActionListener() {
116         @Override
117         public void actionPerformed(java.awt.event.ActionEvent e) {
118             for (int i = 0; i < 12; i++) {
119                 for (int j = 0; j < 5; j++) {
120                     if (e.getSource() == window28.button3[1 + i][1 + j]) {
121                         chosenTest = -1;
122                         for (int k = 0; k < testCounter; k++) {
123                             if (tests[k].getName().equals(window28.button3[1 + i][1 + j].getText())) {
124                                 chosenTest = k;
125                                 window3.field1.setText(tests[chosenTest].getName());
126                                 for (int l = 0; l < 12; l++) {
127                                     if (tests[chosenTest].getSubject().equals(subjects[l])) {
128                                         window3.combol[0].setSelectedIndex(l);
129                                     }
130                                 }
131                                 for (int m = 0; m < groupCounter; m++) {
132                                     if (tests[chosenTest].getGroup().getName().equals(groups[m].getName())) {
133                                         window3.combol[1].setSelectedItem(gNames[m]);
134                                         window3.combol[1].setSelectedIndex(m);
135                                         var = m;
136                                     }
137                                 }
138                             }
139                         }
140                         if (chosenTest == -1) {
141                             window3.field1.setText("");
142                             window3.combol[0].setSelectedIndex(0);
143                             window3.combol[1].setSelectedItem(gNames[0]);
144                         }
145                     }
146                 }
147             }
148         }
149     };
150     for (int i = 0; i < 12; i++) {
151         for (int j = 0; j < 5; j++) {
152             window28.button3[1 + i][1 + j].addActionListener(listener2);
153         }
154     }
155 }
156 }
```

The first 2 loops iterate over the clickable button array `button3` in the calendar panel.

37

```
public JButton[][] button3 = new JButton[13][6];
```

`i` and `j` iterate from 0 to 12 and 0 to 5, respectively and the button `button3[1 + j][1 + i]` is accessed because the first row and column of `button3` are just labels. The flag `chosenTest` is set to -1. If it remains so, the components load empty. Meanwhile,

another loop iterates over the tests array to check to what (if any) test the button corresponds. If a test is found, the test name is loaded and further 2 loops find the test subject and group to be loaded. Finally, the actionListeners are added to the button3 array.

3. Flags

Flags in the program include: `csFlag`, `testCounter` and `groupCounter`. Let's focus on `csFlag`.

```
46 public static boolean csFlag = false;
```

`csFlag` is used in a fragment of the `update()` method that loads the contents of the attendance panel.

```
457     if (csFlag == true) {
458         for (int i = 0; i < 12; i++) {
459             for (int j = 0; j < 5; j++) {
460                 if (chosenStudent.getAttendance()[i][j + window27.counter] == 1) {
461                     window27.button3[1 + i][1 + j].setBackground(Color.GREEN);
462                 } else {
463                     window27.button3[1 + i][1 + j].setBackground(Color.RED);
464                 }
465             }
466         }
467     }
```

In the example, we iterate over the attendance of `chosenStudent` and the corresponding array `button3`. `window27.counter` is used to indicate the week considered. It changes upon clicking the left/right arrow. If attendance = 1, the button's color is set to green, otherwise to red. Since the object `chosenStudent` is accessed, we need to check if it exists to avoid `NullPointerException`. We do this via `csFlag`.

attendance

Trevor

week 1

	monday	tuesday	wednesday	thursday	friday
period 0					
period 1					
period 2					
period 3					
period 4					
period 5					
period 6					
period 7					
period 8					
period 9					
period 10					
period 11					

← → return

A fragment of the `changePanel` method sets `csFlag` back to `false` when we exit the attendance panel and no student is selected anymore.

```

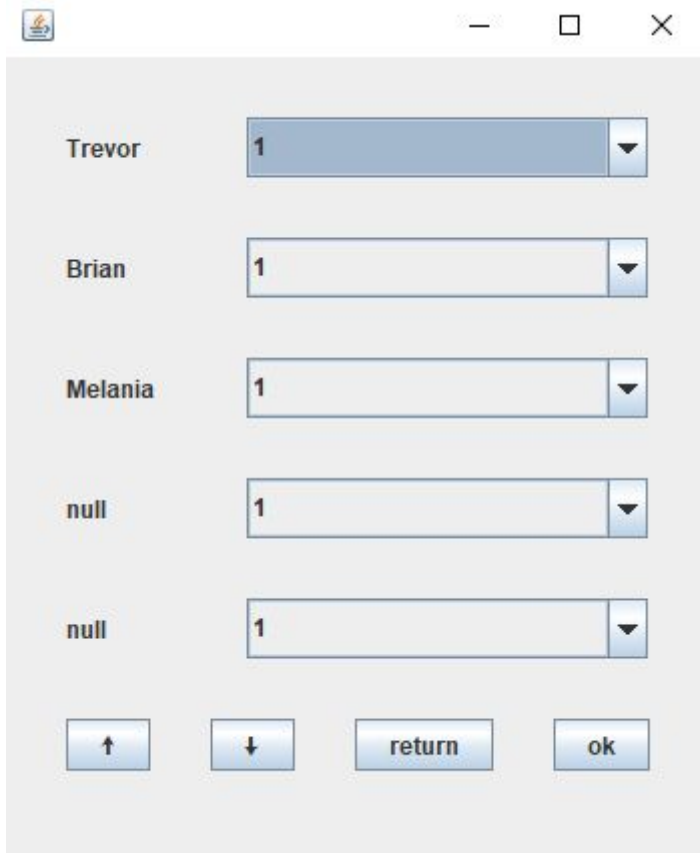
837         if (e.getSource() == window27.button2) {
838             window27.counter = 0;
839             csFlag = false;
840             window27.labell[1].setText("week " + ((window27.counter / 5) + 1));
841             window27.labell[0].setText("student");
            }

```

The fragment also resets the week counter `window27.counter` and the labels.

4. Complex selection

Complex selection is used in the `changePanel` and `update` methods in the main class but also in the `changeComponents()` method in `Panel4` class. `Panel4` serves as a blueprint for the “mark test” panel.



The screenshot shows a Java Swing window titled "Complex selection" with standard window controls (minimize, maximize, close). The window contains a list of five items, each with a text label and a selection box. The selection boxes are currently set to "1". At the bottom of the window, there are four buttons: "up" (with an upward arrow), "down" (with a downward arrow), "return", and "ok".

Name	Selection
Trevor	1
Brian	1
Melania	1
null	1
null	1

Buttons: up, down, return, ok

`changeComponents()` is presented below.

```

33     public int counter; //index of the top student
34     public int[] array = new int[30];
35
36     public void changeComponents() {
37         ActionListener listener = new ActionListener() {
38             @Override
39             public void actionPerformed(java.awt.event.ActionEvent e) {
40                 for (int i = 0; i < 5; i++) {
41                     array[counter + i] = combol[i].getSelectedIndex() + 1;
42                     combol[i].setSelectedIndex(0);
43                 }
44                 if (chosenGroup != -1) {
45                     if (e.getSource() == button2[0] && (counter - 5) >= 0) {
46                         counter = counter - 5;
47                     } else if (e.getSource() == button2[1] && (counter + 5) <= groups[chosenGroup].getCardinality() - 1) {
48                         counter = counter + 5;
49                     }
50                     for (int i = 0; i < 5; i++) {
51                         if ((counter + i) < groups[chosenGroup].getCardinality()) {
52                             labell[i].setText(groups[chosenGroup].getStudents()[counter + i].getName());
53                             if (array[counter + i] != -1) {
54                                 combol[i].setSelectedIndex(array[counter + i] - 1);
55                             }
56                         } else {
57                             labell[i].setText("null");
58                         }
59                     }
60                 }
61             }
62         };
63         button2[0].addActionListener(listener);
64         button2[1].addActionListener(listener);
65     }
66 }

```

First, the method iterates over the 5 comboBoxes, and stores the selected grades in array and resets their selection. Then the flag chosenGroup is checked since groups[chosenGroup] is accessed later on. Then the source is checked and the counter either increases, decreases or doesn't change. For the counter to increase, the index of the next top student has to be smaller or equal to the index of the last student. The subsequent loop iterates over the components and either fills them with student details or leaves them blank if no student matches a given label and comboBox. Finally, listeners are added to the buttons.

5. File i/o

The program reads from and writes to file login details, and attendance, groups and tests. Test class has the following attributes.

```
12 public class Test {  
13  
14     private String name;  
15     private Group group;  
16     private long day;  
17     private int period;  
18     private Mark[] marks;  
19     private String subject;  
20     public boolean marked = false;
```

Array tests is saved to file with the saveCalendar() method.

```
313 public static void saveCalendar() {  
314     String text = "";  
315     String text2 = "";  
316     for (int i = 0; i < testCounter; i++) {  
317         String storr = "";  
318         if (tests[i].marked == true) {  
319             for (int j = 0; j < tests[i].getMarks().length; j++) {  
320                 storr = storr + tests[i].getMarks()[j].getNumber() + " ";  
321             }  
322             text2 = text2 + Integer.toString(i)  
323                 + " " + tests[i].getName()  
324                 + " " + tests[i].getGroup().getName()  
325                 + " " + Long.toString(tests[i].getDay())  
326                 + " " + Integer.toString(tests[i].getPeriod())  
327                 + " " + tests[i].getSubject()  
328                 + " " + tests[i].getMarks().length  
329                 + " " + storr + " ";  
330         } else {  
331             text = text + Integer.toString(i)  
332                 + " " + tests[i].getName()  
333                 + " " + tests[i].getGroup().getName()  
334                 + " " + Long.toString(tests[i].getDay())  
335                 + " " + Integer.toString(tests[i].getPeriod())  
336                 + " " + tests[i].getSubject() + " ";  
337         }  
338     }  
339     writeFile("calendar.txt", text);  
340     writeFile("testCounter.txt", Integer.toString(testCounter));  
341     writeFile("calendarMarked.txt", text2);  
342 }
```

We iterate over the tests array and, if a test is marked, over the tests[i].getMarks array and save the individual marks in stor. Later, text, text2 and testCounter are saved in separate files. Since the number of elements needed to fully describe a marked test changes depending on the size of the group, a longer regular expression “ ” is used to separate the tests. The testCounter is also saved to initialize the tests array when reading the file. The writeToFile method was taken from w3schools.com¹.

¹ W3schools.com. (2020). Java Create and Write To Files. [online] Available at: https://www.w3schools.com/java/java_files_create.asp [Accessed 23 Feb. 2020].

6. Additional libraries

In order to encrypt the login and password, additional libraries presented below have been implemented.

```
16 import java.security.NoSuchAlgorithmException;
17 import java.security.SecureRandom;
18 import java.security.spec.InvalidKeySpecException;
19 import java.security.spec.KeySpec;
20 import javax.crypto.SecretKeyFactory;
21 import javax.crypto.spec.PBEKeySpec;
```

They have been used in the encodeText method taken from baeldung.com².

```
76 public static byte[] encodeText(String text, int i) throws InvalidKeySpecException, NoSuchAlgorithmException {
77     SecureRandom random = new SecureRandom();
78     byte[] salt = new byte[16];
79     random.nextBytes(salt);
80     switch (i) {
81         case 0: //we save actual login
82             lSalt = salt;
83             break;
84         case 1: //we save actual password
85             pSalt = salt;
86             break;
87         case 2: //we save inputted login
88             salt = lSalt;
89             break;
90         case 3: //we save inputted password
91             salt = pSalt;
92             break;
93         default:
94             break;
95     }
96
97     KeySpec spec = new PBEKeySpec(text.toCharArray(), salt, 65536, 128);
98     SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
99
100     byte[] hash = factory.generateSecret(spec).getEncoded();
101     return hash;
102 }
```

I have used the throws clause `InvalidKeySpecException` in the case that `KeySpec spec` is initialized with errors. `NoSuchAlgorithmException` is thrown when the cryptographic algorithm `SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1")` is not found in the environment. First, a byte array salt is generated, which is then filled with random bytes. Later the cryptographic key spec for the parameter text using the

² Millington, S. (2020). [online] Baeldung.com. Available at: <https://www.baeldung.com/java-password-hashing> [Accessed 23 Feb. 2020].

random salt is created. Then, with the use of the `SecretKeyFactory` "PBKDF2WithHmacSHA1" and the `getEncoded()` method, the key is turned into the key specification hash.

Hash is then converted to `String` and saved to file. First, login and password are saved to files using unique salts. When a user inputs them in the login panel, they are also encoded using the same salts as the originals, converted to `String` and compared³. That's why I have used a case/switch in the `encodeText` method.

Bibliography

1. Millington, S. (2020). [online] Baeldung.com. Available at: <https://www.baeldung.com/java-password-hashing> [Accessed 23 Feb. 2020].
2. Stack Overflow. (2020). Java: how to write/read encrypted password in/from a file. [online] Available at: <https://stackoverflow.com/questions/27864072/java-how-to-write-read-encrypted-password-in-from-a-file> [Accessed 23 Feb. 2020].
3. W3schools.com. (2020). Java Create and Write To Files. [online] Available at: https://www.w3schools.com/java/java_files_create.asp [Accessed 23 Feb. 2020].

Word count: 1008

³ Stack Overflow. (2020). Java: how to write/read encrypted password in/from a file. [online] Available at: <https://stackoverflow.com/questions/27864072/java-how-to-write-read-encrypted-password-in-from-a-file> [Accessed 23 Feb. 2020].